

Министерство образования Республики Беларусь
Учреждение образования
Белорусский государственный университет информатики и
Радиоэлектроники

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2
по курсу “Логические основы интеллектуальных систем”
Вариант 3

Выполнил:
Студент гр. 421702

Евик А.Н.

Проверил:

Ивашенко В.П.

Минск

2026

Содержание

1. Грамматика языка PROLOG.....	3
2. Реализация.....	4
3. Листинг программы.....	8
4. Формализация основных фактов и правил на языке логики предикатов первого порядка.....	12
5. Дерево формального логического вывода.....	12
5. Результаты тестирования.....	13
6. Вывод.....	15
Список использованных источников.....	16

Тема:

Решение логических задач на языке Prolog.

Цель:

В соответствии с вариантом необходимо реализовать программу на языке Prolog, решающую поставленную задачу.

Задание:

Два берега реки. На одном из берегов есть три семейные пары, требуется с помощью лодки, вмещающей не более двух человек, переправить всех на другой берег. Нельзя оставлять чужую жену вместе с чужим мужем без присмотра своего супруга.

Дополнительные теоретические сведения:

Prolog – средство написания выполняемых на ЭВМ программ. Язык логического программирования, основанный на логике предикатов первого порядка.

1. Грамматика языка PROLOG

$\langle \text{ПРОЛОГ-предложение} \rangle ::= \langle \text{правило} \rangle \mid \langle \text{факт} \rangle \mid \langle \text{запрос} \rangle$

$\langle \text{правило} \rangle ::= \langle \text{заголовок} \rangle \text{ ‘:-’ } \langle \text{тело} \rangle$

$\langle \text{факт} \rangle ::= \langle \text{заголовок} \rangle \text{ ‘.’}$

$\langle \text{запрос} \rangle ::= \langle \text{тело} \rangle \text{ ‘.’}$

$\langle \text{тело} \rangle ::= \langle \text{цель} \rangle \text{ /’,’ } \langle \text{цель} \rangle \text{ /’.’}$

$\langle \text{заголовок} \rangle ::= \langle \text{предикат} \rangle$

$\langle \text{цель} \rangle ::= \langle \text{предикат} \rangle \mid \langle \text{выражение} \rangle$

$\langle \text{предикат} \rangle ::= \langle \text{имя} \rangle \text{ / (‘ } \langle \text{терм} \rangle \text{ /’,’ } \langle \text{терм} \rangle \text{ / ’) /}$

$\langle \text{терм} \rangle ::= \langle \text{атом} \rangle \mid \langle \text{предикат} \rangle \mid \langle \text{список} \rangle$

$\langle \text{атом} \rangle ::= \langle \text{переменная} \rangle \mid \langle \text{число} \rangle \mid \langle \text{строка} \rangle \mid \langle \text{имя} \rangle$

$\langle \text{список} \rangle ::= \langle \text{список с заголовком} \rangle \mid \langle \text{простой список} \rangle$

<список с заголовком>::= '[' <терм>/','<терм>/']' <терм>']'

<простой список>::= '[' <терм>/','<терм>/']' '['']'

<выражение>::= <терм> /<оператор><терм>/

<оператор>::= 'is' | '=' | '==' | '\=' | '>=' | '<=' | '=\\=' |

2. Реализация

Для решения задачи использовался SWI-Prolog Online (SWISH). Для представления состояния задачи будет использоваться функциональный терм `state(list, pos)`, в котором первый параметр определяет список людей на левом берегу второй параметр - местоположение плота. Pos может принимать значения `left` или `right`, что означает нахождение плота у левого или правого берега соответственно. Таким образом, `state([man1, man2, woman1, woman2 ], left)` означает, что плот сейчас находится у левого берега, на левом берегу находится две семейные пары, тогда на правом берегу находятся третья пара. Тогда терминальным состоянием будет `state([], any)`.

В программе используются:

- Встроенные предикаты

- `forall(Cond, Action)`.

Для всех возможных Cond, Action может быть доказано.

- `member (X, List)`.

Истинно, если X является членом списка List.

- `subtract(Source, Subtrahend, Result)`.

Вычитает из списка Source содержимое списка Subtrahend и записывает результат в список Result.

- `sort(Source_list, Destination_list)`.

Истинно, если результат сортировки в обычном порядке элементов списка Source_list может быть унифицирован со списком Destination_list. При этом все повторяющиеся элементы списка Source_list удаляются в полученном после сортировки результате.

- `format(Format_str, Arguments)`.

Печатает содержимое Format_str, подставляя на указанные места соответствующие элементы списка Arguments.

- Action1; Action2.
Дизъюнкция. Возвращает истину, если истинно хотя бы одно из [Action1, Action2].
- Condition -> Action.
Импликация. Action выполняется, если истинно Condition.
- !
Отсечение. Применяется, если хватает одного решения, если программист знает о единственности решения, если корректно лишь первое решение и нужно удалить другие решения.
- =
Достигается вместе с унификацией и может повлечь подстановку.
- \+
Описывает логическое отрицание.
- :-
Описывает логическое "если", и служит для разделения двух частей правила: заголовка и тела.

- **Факты**

- woman(woman1). – факт, утверждающий, что woman1 является женщиной.
- woman(woman2). – факт, утверждающий, что woman2 является женщиной.
- woman(woman3). – факт, утверждающий, что woman3 является женщиной.
- man(man1). – факт, утверждающий, что man1 является мужчиной
- man(man2). – факт, утверждающий, что man2 является мужчиной.
- man(man3). – факт, утверждающий, что man3 является мужчиной.
- family(man1, woman1). – факт, утверждающий, что man1 и woman1 являются семейной парой.
- family(man2, woman2). – факт, утверждающий, что man2 и woman2 являются семейной парой.
- family(man3, woman3). – факт, утверждающий, что man3 и woman3 являются семейной парой.
- all([man1, woman1, man2, woman2, man3, woman3]). – список всех людей.
- allsafe([_]). – факт, который возвращает истину, если на берегу находится один человек.

- `allsafe([])`. – факт, который возвращает истину, если на берегу нет людей.

- Правила

- `notsafe_(X, Y) :- woman(Y), man(X), \+ family(X, Y)`. – истинно, если X и Y не являются семейной парой.
- `notsafe(X, Y) :- notsafe_(X, Y); notsafe_(Y, X)`. – истинно, если `notsafe_(X, Y)` или `notsafe_(Y, X)`.
- `safe(X, Y) :- \+ notsafe(X, Y)`. – истинно, если не `notsafe(X, Y)`.
- `safebridge([X, Y]) :- safe(X, Y), !`. – истинно, если `safe(X, Y)`.
- `safebridge([X]) :- man(X); woman(X)`. – истинно, если `man(X)` или `woman(X)`.
- `allPairs([H | T], [H, P2]) :- member(P2, T)`.

Первым и вторым аргументами принимает списки. Во второй список запишется результат, состоящий из двух элементов: H – голова первого списка и P2 – элемент хвоста T первого списка.

- `allPairs([_ | T], P) :- allPairs(T, P)`.

Первым аргументом и вторым аргументами принимает списки. Во второй список запишет результат `allPairs` для хвоста первого списка. В совокупности с предыдущим правилом `allPairs` позволяет перебрать все возможные пары элементов в каком-либо списке (используются для определения пар, которые поместятся на плоту).

- `allsafe(L) :-`
 `forall(`
 `member(H, L),`
 (
 `woman(H),`
 (
 (
 `man(X),`
 `family(X, H),`
 `member(X, L)`
);
 `\+`
 (
 `member(M, L),`
 `man(M)`

```

    )
  );
  man(H)
)
), !.

```

Истинно, если для всех элементов L выполняется: H – женщина и есть такой мужчина X, что X и H – семейная пара или вовсе нету мужчин,
или H – мужчина.

- `step(state(Left1, left), state(Left2, right)) :- step_(state(Left1, left), state(Left2, right)).`

Определяет следующий шаг с помощью `step_` в ситуации, когда плот находится на левом берегу.

- `step(state(Left1, right), state(Left2, left)) :-
 all(All),
 subtract(All, Left1, Right1),
 step_(state(Right1, left), state(Right2, right)),
 subtract(All, Right2, Left2).`

Определяет следующий шаг с помощью `step_` в ситуации, когда плот находится на правом берегу. После чего определяет кто остался на левом берегу.

- `step_(state(Left1, left), state(Left2, right)) :-
 (
 allPairs(Left1, OnBridge);
 member(A, Left1),
 OnBridge = [A]
),
 safebridge(OnBridge),

 subtract(Left1, OnBridge, Left2),
 allsafe(Left2),

 all(All),`

```
subtract(All, Left2, Right),
allsafe(Right).
```

Истинно, если на берегу можно найти такую пару(или одного человека), которая (который) могла (мог) бы переправиться на противоположный берег, при выполнении правил safebridge, для пары (человека), переправляющихся (отправляющегося) на плоту, при выполнении правила allsafe() для людей на левом и правом берегах после переправы.

- `notequal(state(L1, P1), state(L2, P2)) :-`
 `\+ (`
 `P1 = P2,`
 `sort(L1, L),`
 `sort(L2, L)`
 `).`

Истинно, если `state(L1, P1)` не эквивалентен `state(L2, P2)` (если списки состоят из разных элементов или плот находится на разных берегах).

- `path(Inp, PrevSteps, [step(Inp, S1) | Steps]) :-`
 `step(Inp, S1),`
 `duplCheck(S1, PrevSteps),`
 `path(S1, [step(Inp, S1) | PrevSteps], Steps).`

Истинно, если существует путь от состояния `Inp`, до терминального состояния `state([], _)`, является целью задачи.

3.Листинг программы

```
:- encoding(utf8).
:- set_prolog_flag(encoding, utf8).

% Лабораторная работа № 2 по дисциплине "Логические основы
интеллектуальных систем"
% Выполнена студентом ВГУИР Евик Алексей Николаевич
% Условие: Два берега реки. На одном из берегов есть три семейные
пары, требуется с помощью лодки, вмещающий
% не более двух человек, переправить всех на другой берег. Нельзя
оставлять чужую жену и чужого мужа
% вместе без супругов.

family(man1, woman1).
family(man2, woman2).
```



```

family(man3, woman3).

woman(woman1).
woman(woman2).
woman(woman3).

man(man1).
man(man2).
man(man3).

notsafe_(X, Y) :-
    woman(Y),
    man(X),
    \+ family(X, Y).

notsafe(X, Y) :- notsafe_(X, Y); notsafe_(Y, X).

safe(X, Y) :- \+ notsafe(X, Y).

safebridge([X, Y]) :-
    safe(X, Y),
    !.

safebridge([X]) :-
    man(X);
    woman(X).

all([
    man1, woman1,
    man2, woman2,
    man3, woman3
]).

allsafe(L) :-
    forall(
        member(H, L),
        (
            woman(H),
            (
                (
                    man(X),
                    family(X, H),
                    member(X, L)
                );
                \+
                (
                    member(M, L),
                    man(M)
                )
            )
        );
        man(H)
    ), !.

```

```

allsafe([_]).
allsafe([]).

allPairs([H | T], [H, P2]) :-
    member(P2, T).

allPairs([_ | T], P) :-
    allPairs(T, P).

step_(state(Left1, left), state(Left2, right)) :-
    (
        allPairs(Left1, OnBridge);
        member(A, Left1),
        OnBridge = [A]
    ),
    safebridge(OnBridge),

    subtract(Left1, OnBridge, Left2),
    allsafe(Left2),

    all(All),
    subtract(All, Left2, Right),
    allsafe(Right).

step(state(Left1, left), state(Left2, right)) :-
    step_(state(Left1, left), state(Left2, right)).

step(state(Left1, right), state(Left2, left)) :-
    all(All),
    subtract(All, Left1, Right1),
    step_(state(Right1, left), state(Right2, right)),
    subtract(All, Right2, Left2).

notequal(state(L1, P1), state(L2, P2)) :-
    \+ (
        P1 = P2,
        sort(L1, L),
        sort(L2, L)
    ).

duplCheck(_, []).
duplCheck(CurrState, [step(State1, _) | T]) :-
    notequal(CurrState, State1),
    duplCheck(CurrState, T).

path(state([], _), _, []).

path(Inp, PrevSteps, [step(Inp, S1) | Steps]) :-
    step(Inp, S1),

```

```

    duplCheck(S1, PrevSteps),
    path(S1, [step(Inp, S1) | PrevSteps], Steps).

valid_state(state(Left, Bridge)) :-
    (Bridge == left; Bridge == right),
    all(All),
    subtract(All, Left, Right),
    append(Left, Right, AllCurr),
    sort(All, Tmp),
    sort(AllCurr, Tmp).

printStep(step(state(L1, Pos1), state(L2, _))) :-
    ( Pos1 = left
    -> subtract(L1, L2, Moved),
        all(All),
        subtract(All, L1, R1),
        format('left: ~w | right: ~w~n', [L1, R1]),
        subtract(All, L1, R2),
        format('left: ~w | raft: ~w -> | right: ~w~n~n', [L2, Moved,
R2]));
        Pos1 = right
    -> subtract(L2, L1, Moved),
        all(All),
        subtract(All, L1, R1),
        format('left: ~w | right: ~w~n', [L1, R1]),
        subtract(All, L2, R2),
        format('left: ~w | raft: ~w <- | right: ~w~n~n', [L1, Moved,
R2])
    ).

solve(StartLeft, StartBridge):-
    valid_state(state(StartLeft, StartBridge)),

    sort(StartLeft, SortedStartLeft),
    all(All),

    allsafe(SortedStartLeft),
    subtract(All, SortedStartLeft, StartRight),

    allsafe(StartRight),
    path(state(SortedStartLeft, StartBridge), [], Steps),
    (
        format('~nSolution:~n'),
        forall(member(Step, Steps), printStep(Step))
    ).

```

4. Формализация основных фактов и правил на языке логики предикатов первого порядка

$M(x)$ - x является мужчиной;

$W(y)$ - y является женщиной;

$F(x, y)$ - x и y являются супругами;

$N(x, y)$ - состояние x и y небезопасно;

$O(x, b, s)$ - x находится на берегу b в состоянии s ;

$B(b, s)$ - лодка на берегу b в состоянии s ;

$S(b, s)$ - берег b в состоянии s небезопасен

$N(s1, s2)$ - состояние $s2$ достижимо из состояния $s1$ за один шаг.

Безопасность берега b в состоянии s :

$(\forall b(\forall s(\forall w((W(w)\wedge O(w,b,s))\rightarrow((\exists m(F(m,w)\wedge O(m,b,s)))\vee(!\exists r(M(r)\wedge O(r,b,s)))))))$
 $))$

Безопасность пары в лодке:

$(\forall x(\forall y(S(x,y)\sim(!((W(x)\wedge M(y))\wedge(!F(y,x))))\vee((W(y)\wedge M(x))\wedge(!F(x,y))))))$

Переход из состояния $s1$ в $s2$:

$(\forall s(\forall r(N(s,r)\sim(\exists t(\exists u((B(t,s)\wedge B(u,r)))\wedge(\exists x(\exists y((O(x,t,s)\wedge O(y,t,s))\wedge((O(x,u,r)\wedge O(y,u,r))\wedge(S(x,y)\wedge(S(t,r)\wedge S(u,r))))))))))))$

5. Дерево формального логического вывода

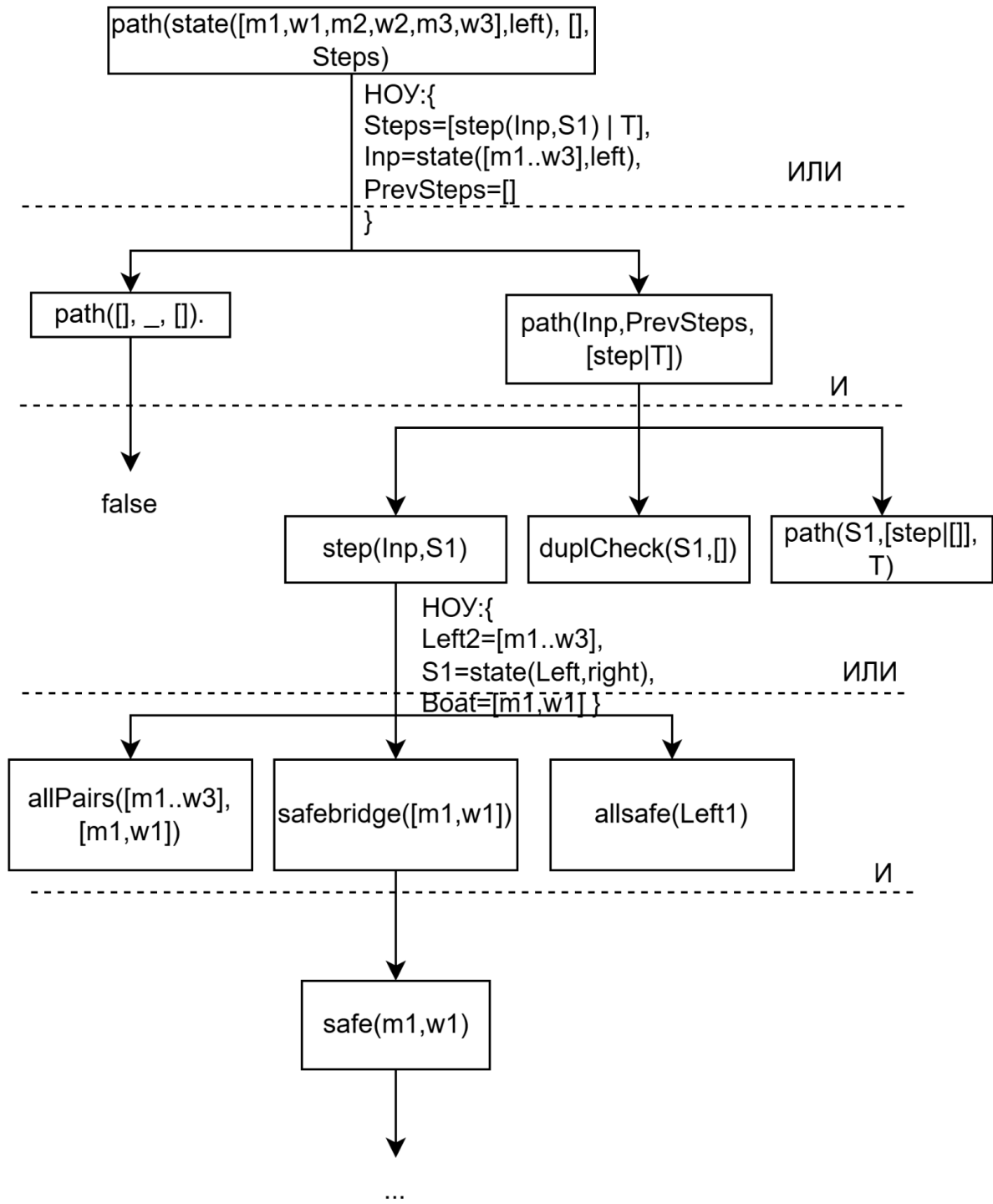


Рис. 5.1 - Дерево формального логического вывода

5. Результаты тестирования

Тестирование выполнено в среде SWI-Prolog Online (SWISH). Ниже приведены примеры исполнения программы.

Запуск программы дает решение из 11 переправ:

```
left: [man1,man2,man3,woman1,woman2,woman3] | right: []  
left: [man2,man3,woman2,woman3] | raft: [man1,woman1] -> | right: []  
  
left: [man2,man3,woman2,woman3] | right: [man1,woman1]  
left: [man2,man3,woman2,woman3] | raft: [man1] <- | right: [woman1]  
  
left: [man1,man2,woman2,man3,woman3] | right: [woman1]  
left: [man1,man2,man3] | raft: [woman2,woman3] -> | right: [woman1]  
  
left: [man1,man2,man3] | right: [woman1,woman2,woman3]  
left: [man1,man2,man3] | raft: [woman1] <- | right: [woman2,woman3]  
  
left: [man1,woman1,man2,man3] | right: [woman2,woman3]  
left: [man1,woman1] | raft: [man2,man3] -> | right: [woman2,woman3]  
  
left: [man1,woman1] | right: [man2,woman2,man3,woman3]  
left: [man1,woman1] | raft: [man2,woman2] <- | right: [man3,woman3]  
  
left: [man1,woman1,man2,woman2] | right: [man3,woman3]  
left: [man2,woman2] | raft: [man1,woman1] -> | right: [man3,woman3]  
  
left: [man2,woman2] | right: [man1,woman1,man3,woman3]  
left: [man2,woman2] | raft: [man3,woman3] <- | right: [man1,woman1]
```

```

left: [man2,woman2,man3,woman3] | right: [man1,woman1]

left: [man3,woman3] | raft: [man2,woman2] -> | right: [man1,woman1]


left: [man3,woman3] | right: [man1,woman1,man2,woman2]

left: [man3,woman3] | raft: [man1,woman1] <- | right: [man2,woman2]


left: [man1,woman1,man3,woman3] | right: [man2,woman2]

left: [woman1,woman3] | raft: [man1,man3] -> | right: [man2,woman2]


left: [woman1,woman3] | right: [man1,man2,woman2,man3]

left: [woman1,woman3] | raft: [woman2] <- | right: [man1,man2,man3]


left: [woman1,woman2,woman3] | right: [man1,man2,man3]

left: [woman3] | raft: [woman1,woman2] -> | right: [man1,man2,man3]


left: [woman3] | right: [man1,woman1,man2,woman2,man3]

left: [woman3] | raft: [woman1] <- | right: [man1,man2,woman2,man3]


left: [woman1,woman3] | right: [man1,man2,woman2,man3]

left: [] | raft: [woman1,woman3] -> | right: [man1,man2,woman2,man3]

true

```

6. Вывод

В ходе лабораторной работы реализована программа на языке Prolog, решающая задачу о супружеских парах (вариант 3) посредством поиска в пространстве состояний. Состояние представлено списками именованных персон, что обеспечивает корректный учет ограничения, зависящего от тождества персон. Применен поиск в ширину, гарантирующий минимальность числа переправ.

Список использованных источников

1. Голенков, В. В. Логические основы интеллектуальных систем. Практикум : учеб.-метод. пособие / В. В. Голенков [и др.]. – Минск : БГУИР, 2011. – 70 с.
2. SWI-Prolog reference manual [сайт]. - Режим доступа: <https://www.swi-prolog.org>. (дата доступа: 27.05.2026)
3. Решение логических задач на PROLOG [сайт]. - Режим доступа: <https://pro-prof.com/archives/1299>. (дата доступа: 27.05.2026)